TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN
ĐHQG-HCM
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN 1

Ma Trận và Cơ Sở của Tính Toán Khoa Học

MÔN: Toán Ứng dụng và Thống kê

Lớp: 24CTT3

Nhóm 3

Tác giả:

24120023 - Nguyễn Thành Bảo
24120044 - Bùi Thị Thùy Dương
24120055 - Nguyễn Khánh Hoàng
24120085 - Lê Nguyễn Thùy Linh
24120094 - Mai Anh Phúc Minh

Giảng viên hướng dẫn:

ThS. Võ Nam Thục Đoan
ThS. Lê Nhựt Nam

Tài liệu được thực hiện bằng LaTeX

Mục lục

Lời mở đầu **3**

Phần 1: Phép khử Gauss và Các Ứng Dụng **4**

1. Cơ Sở Lý Thuyết 4
   - 1.1. Ma trận Dạng bậc thang (REF) và Dạng bậc thang rút gọn (RREF) 4
   - 1.2. Kĩ thuật chọn Partial Pivoting trong phép khử Gauss 4
   - 1.3. Phương pháp khử Gauss 4
   - 1.4. Phương pháp khử Gauss–Jordan 5
   - 1.5. Tính định thức qua Phép khử Gauss 5
   - 1.6. Tìm ma trận nghịch đảo 5
   - 1.7. Hạng và cơ sở của ma trận 5
   - 1.8. Nghiệm của một hệ phương trình 6
2. Mô Tả Chi Tiết Tất Cả Các Hàm Trong `part1.py` 6
   - 2.1. gaussian_eliminate(A, b) 6
   - 2.2. backward_elimination(M, pivot_cols) 8
   - 2.3. solve_infinite_solutions(M_rref, pivot_cols, m_vars, ϵ = 1e-12) 9
   - 2.4. solver(A, b) 11
   - 2.5. back_substitution(U, c) 12
   - 2.6. determinant(A) 13
   - 2.7. inverse(A) 14
   - 2.8. rank_and_basis(A) 15
   - 2.9. verify_solutions(A, x, b) 16
   - 2.10. Các hàm còn lại 17

Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim **18**

1. Chéo hóa ma trận 18
   - 1.1. Cơ sở lý thuyết 18
   - 1.2. Mô tả chi tiết tất cả các hàm trong `diagonalization.py` 19
   - 1.3. Bảng Tổng Hợp Tất Cả Các Hàm 26
   - 1.4. Phân tích ưu điểm, hạn chế và độ phức tạp 26
   - 1.5. So sánh kết quả với thư viện NumPy 27
   - 1.6. Ứng dụng chéo hóa trong thực tế 28
2. Phân rã Cholesky (Cholesky Decomposition) 28
   - 2.1. Cơ sở lý thuyết 28
   - 2.2. Thuật toán phân rã 28
   - 2.3. Cài đặt thuật toán 30
   - 2.4. Minh họa mã nguồn (Source Code) 30
   - 2.5. Kết quả và Kiểm thử (Unit Testing) 31
   - 2.6. Ứng dụng thực tiễn của Phân rã Cholesky 32
3. Trực quan hóa bằng Manim 33

Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng **34**

1. Tổng Quan Yêu Cầu 34
2. Lý Thuyết Sai Số và Tính Ổn Định Số 34
2.1. Số Điều Kiện (Condition Number) 34
2.2. So Sánh Các Phương Pháp 34
2.3. Phương Pháp Lặp Gauss–Seidel 35
3. Thực Nghiệm Thời Gian và Độ Phức Tạp (Benchmark) 35
3.1. Cài đặt thực nghiệm 35
3.2. Kết quả thực nghiệm 35
3.3. Nhận xét Benchmark 35
4. Phân Tích Ổn Định Số 37
4.1. Cài đặt thực nghiệm phân tích ổn định 37
4.2. Nhận xét Phân Tích Ổn Định Số 37
5. Kết Luận Cuối Cùng 38

Kết luận **39**

Bảng phân chia công việc **39**

Lời cảm ơn **40**

Tài liệu tham khảo **40**

Lời mở đầu

Trong bối cảnh công nghệ thông tin ngày càng phát triển mạnh mẽ và đóng vai trò quan trọng trong hầu hết các lĩnh vực của đời sống, việc áp dụng các phương pháp tính toán và thuật toán vào giải quyết các bài toán thực tiễn trở nên vô cùng cần thiết. Đặc biệt, trong các ngành liên quan đến khoa học máy tính, toán ứng dụng và kỹ thuật phần mềm, việc xây dựng các giải thuật hiệu quả không chỉ giúp tối ưu hóa thời gian xử lý mà còn nâng cao độ chính xác của kết quả.

Bài toán hệ phương trình tuyến tính là một trong những bài toán cơ bản nhưng có ý nghĩa nền tảng trong nhiều lĩnh vực như đồ họa máy tính, xử lý tín hiệu, học máy, tối ưu hóa và mô phỏng hệ thống. Việc giải hệ phương trình bằng phương pháp thủ công thường chỉ phù hợp với các hệ nhỏ, trong khi các hệ lớn đòi hỏi phải có sự hỗ trợ của thuật toán và máy tính để đảm bảo tính khả thi và hiệu quả.

Trong khuôn khổ môn học này, nhóm chúng em thực hiện xây dựng và cài đặt thuật toán khử Gauss (Gaussian Elimination) kết hợp với phương pháp chọn phần tử chính (Partial Pivoting). Thuật toán này giúp đưa ma trận hệ số về dạng bậc thang, từ đó giải hệ phương trình tuyến tính một cách hệ thống và chính xác hơn. Đồng thời, chương trình còn được mở rộng để kiểm tra nghiệm, tính định thức và xác định hạng của ma trận, nhằm cung cấp cái nhìn toàn diện hơn về hệ phương trình.

Thông qua việc thực hiện bài tập lớn này, nhóm chúng em không chỉ củng cố lại kiến thức lý thuyết đã học mà còn có cơ hội áp dụng vào thực tế lập trình, từ đó hiểu rõ hơn về cách một thuật toán được triển khai và tối ưu hóa. Đây cũng là dịp để các thành viên trong nhóm rèn luyện kỹ năng làm việc nhóm, phân chia công việc và phối hợp trong quá trình phát triển sản phẩm.

Bên cạnh đó, bài báo cáo này cũng không tránh khỏi những thiếu sót do hạn chế về kinh nghiệm và thời gian thực hiện. Vì vậy, nhóm rất mong nhận được những ý kiến đóng góp từ thầy cô và các bạn để có thể hoàn thiện hơn trong những lần thực hiện sau.

Phần 1: Phép khử Gauss và Các Ứng Dụng

1. Cơ Sở Lý Thuyết

1.1. Ma trận Dạng bậc thang (REF) và Dạng bậc thang rút gọn (RREF)

Định nghĩa:

- Một ma trận gọi là **REF** nếu thỏa mãn các tính chất sau:
  1. Tất cả các hàng chỉ chứa số 0 (nếu có) phải nằm ở dưới cùng của ma trận.
  2. Phần tử khác 0 đầu tiên của mỗi hàng (gọi là **pivot**) phải nằm ở bên phải của phần tử trụ của hàng ngay phía trên nó.
  3. Tất cả các phần tử nằm bên dưới một pivot phải bằng 0.
- Một ma trận gọi là **RREF** nếu thỏa mãn các tính chất sau:
  1. Nó là một ma trận **REF**.
  2. Pivot của mỗi hàng phải là số 1.
  3. Mọi phần tử nằm **phía trên** và **phía dưới** của pivot phải bằng 0.

Ma trận **REF** thường thu được sau phép khử Gauss, trong khi đó ma trận **RREF** thường thu được sau phép khử Gauss–Jordan.

1.2. Kĩ thuật chọn Partial Pivoting trong phép khử Gauss

Trước khi thực hiện bất kỳ bước khử nào, chúng ta cần chọn pivot nhằm tránh việc chia cho số quá nhỏ (gây sai số làm tròn nghiêm trọng) hoặc chia cho 0.

Quy trình thực hiện tại cột j (đang xét hàng i):

1. Tìm phần tử có giá trị tuyệt đối lớn nhất trong cột j, từ hàng i trở xuống:

$$\max_{k=i\ldots n} |a_{kj}|$$

2. Hoán đổi hàng hiện tại (i) với hàng chứa phần tử lớn nhất đó (k).
3. Tiếp tục quá trình khử.

1.3. Phương pháp khử Gauss

Mục tiêu là đưa ma trận hệ số A về dạng ma trận tam giác trên (Upper Triangular Matrix) bằng các phép biến đổi hàng sơ cấp.

Các bước thực hiện:

1. **Forward Elimination (Khử xuôi):** Với mỗi cột từ 1 đến $n-1$:
   - Thực hiện **Partial Pivoting**.
   - Khử các phần tử nằm dưới đường chéo chính để đưa về 0.
2. **Back Substitution (Thế ngược):**
   - Tính $x_n = \dfrac{b'_n}{a'_{nn}}$.
   - Thế x_n vào phương trình phía trên để tìm x_{n-1}, tiếp tục như vậy ngược lên trên cho đến x_1.

1.4. Phương pháp khử Gauss–Jordan

Đây là phiên bản mở rộng của phép khử Gauss, đưa ma trận về dạng ma trận bậc thang rút gọn.

- Tương tự như khử Gauss, nhưng sau khi khử các phần tử dưới đường chéo, ta chia cả hàng cho phần tử chốt để đưa phần tử trên đường chéo về 1.
- Thay vì chỉ dừng lại ở ma trận tam giác trên, ta tiếp tục khử các phần tử **nằm phía trên** đường chéo chính để đưa chúng về 0.

1.5. Tính định thức qua Phép khử Gauss

Sau khi đã đưa ma trận A về REF (bây giờ là ma trận U), ta có thể tính định thức của ma trận này bằng công thức:

$$\det(A) = (-1)^s \cdot \prod_{i=1}^{n} U_{ii}$$

Trong đó:

- s: số lần hoán đổi dòng.
- U_{ii}: các phần tử nằm trên đường chéo chính của ma trận U.

1.6. Tìm ma trận nghịch đảo

Định lý: Ma trận $A \in \mathbb{R}^{n\times n}$ khả nghịch khi và chỉ khi $\det(A) \neq 0$.

Ta có thể tìm A^{-1} bằng cách tạo ma trận tăng cường:

$$M = [A \mid I]$$

Sau đó, thực hiện khử Gauss–Jordan cho ma trận A để đưa về:

$$M' = [I \mid B]$$

Khi đó, ma trận B chính là nghịch đảo của A, tức là:

$$B = A^{-1}$$

1.7. Hạng và cơ sở của ma trận

1. **Hạng (rank):** là số lượng hàng độc lập tuyến tính (hoặc số lượng cột độc lập tuyến tính) của ma trận đó, bằng số dòng khác 0 trong dạng REF của A, hoặc bằng số cột pivot.
2. **Không gian cột (Column Space):** là tập hợp tất cả các tổ hợp tuyến tính của các cột của ma trận A, có số chiều bằng $\mathrm{rank}(A)$. Không gian cột thu được bằng cách:
   - Đưa ma trận về RREF.
   - Xác định các vị trí của các cột chứa pivot.
   - Lấy các cột của ma trận A, tương ứng với các vị trí đã chọn được.
3. **Không gian dòng (Row Space):** là tập hợp tất cả các tổ hợp tuyến tính của các hàng của ma trận A, cũng có số chiều bằng $\mathrm{rank}(A)$; thu được bằng cách lấy các hàng khác 0 của ma trận REF (hoặc RREF).

4. **Không gian nghiệm (Null Space):** là tập hợp tất cả các vector $\mathbf{x}$ thỏa mãn phương trình thuần nhất:

$$A\mathbf{x} = \mathbf{0}$$

Không gian nghiệm thu được bằng cách giải hệ phương trình:

$$A\mathbf{x} = \mathbf{0}$$

1.8. Nghiệm của một hệ phương trình

Nếu hệ phương trình $A\mathbf{x} = \mathbf{b}$ có nghiệm, nghiệm này có thể được biểu diễn dưới dạng:

$$\mathbf{x} = \mathbf{x}_p + \mathbf{x}_n$$

Trong đó:

- $\mathbf{x}_p$ **(Nghiệm riêng – Particular):** là một nghiệm cụ thể của hệ, thu được bằng cách gán cho tất cả các biến tự do bằng 0.
- $\mathbf{x}_n$ **(Nghiệm thuần nhất – Null Space Solution):** là tập hợp tất cả các nghiệm của hệ $A\mathbf{x} = \mathbf{0}$. Đây chính là tổ hợp tuyến tính của các vector cơ sở của không gian nghiệm:

$$\mathbf{x}_n = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_k\mathbf{v}_k$$

Nếu phương trình $A\mathbf{x} = \mathbf{b}$ có nghiệm duy nhất, ta sẽ có:

$$\mathbf{x}_n = \mathbf{0}$$

Suy ra:

$$\mathbf{x} = \mathbf{x}_p$$

2. Mô Tả Chi Tiết Tất Cả Các Hàm Trong `part1.py`

2.1. `gaussian_eliminate(A, b)`

Chức năng của hàm:

Đưa ma trận A về ma trận bậc thang theo phép khử Gauss, sử dụng *Partial Pivoting*. Thuật toán Partial Pivoting hoạt động như sau:

Giả sử A có n dòng (tương ứng với n phương trình trong hệ phương trình).

1. Tại bước thứ k (tức là đang ở cột k), ta tìm phần tử có giá trị tuyệt đối lớn nhất trong cột k đó (giả sử phần tử tìm được nằm ở dòng p).
2. Hoán đổi dòng k với dòng p.
3. Thực hiện khử Gauss: với i từ $k+1 \to n$, ta khử Gauss:

$$\text{dòng } i = \text{dòng } i - \text{factor} \times \text{dòng } k, \quad \text{với factor} = \frac{a_{ik}}{a_{kk}}$$

Trong quá trình khử, nếu dòng $i = 0$ thì bỏ qua.

Các tham số truyền vào:

- A: Ma trận ban đầu (vế trái của hệ phương trình).
- b: Ma trận vế phải.

Giải thích code:

1. **Khởi tạo các biến cơ bản**

   Khởi tạo kích thước ma trận:

   $$n = \text{len}(A), \quad m = \text{len}(A[0])$$

   Tạo ma trận mở rộng $M = (A|b)$.

   - b có thể là vector (1 chiều) hoặc ma trận (2 chiều)
   - Chuẩn hóa b về dạng cột
   - Khởi tạo:
     - s: số lần hoán đổi dòng
     - $pivot_cols$: danh sách cột pivot
     - $pivot_row$: dòng hiện tại

```python
n, m = len(A), len(A[0])

if b is not None:
    b_norm = [[v] if not isinstance(v, list) else v[:] for v in b]
else:
    b_norm = [[] for _ in range(n)]

M = [A[i] + b_norm[i] for i in range(n)]

s = 0
pivot_cols = []
pivot_row = 0
```

Listing 1.1: Khởi tạo dữ liệu

2. **Duyệt qua từng cột**

   Với mỗi cột k, thực hiện:

   - Chọn pivot (phần tử lớn nhất theo trị tuyệt đối)
   - Nếu cột toàn 0 thì bỏ qua

```python
if pivot_row >= n:
    break

p = pivot_row
for i in range(pivot_row + 1, n):
    if abs(M[i][k]) > abs(M[p][k]):
        p = i

if abs(M[p][k]) < 1e-12:
```

```
    continue

pivot_cols.append(k)
```

Listing 1.2: Chọn pivot

Hoán đổi dòng:

```
M[pivot_row], M[p] = M[p], M[pivot_row]

if p != pivot_row:
    s += 1
```

Listing 1.3: Hoán đổi dòng

Khử Gauss:

- Tính hệ số:

$$\text{factor} = \frac{M[i][k]}{M[pivot_row][k]}$$

- Cập nhật dòng:

```
for i in range(pivot_row + 1, n):
    factor = M[i][k] / M[pivot_row][k]
    M[i] = M[i] - factor * M[pivot_row]
    M[i][k] = 0
```

Listing 1.4: Khử Gauss

3. **Kết quả**

   - Ma trận bậc thang
   - Có thể dùng để giải hệ hoặc tính rank
   - s: số lần đổi dòng
   - $pivot_cols$: các cột pivot

Độ phức tạp: $O(n \cdot m^2)$, với n, m lần lượt là số dòng và số cột của ma trận.

2.2. `backward_elimination(M, pivot_cols)`

Chức năng của hàm:

Khử ma trận tam giác trên (ma trận đã qua khử Gauss) thành ma trận đường chéo theo phương pháp Gauss-Jordan. Ma trận kết quả được sử dụng để tìm nghiệm tổng quát của hệ vô số nghiệm.

Tham số truyền vào:

- M: Ma trận tam giác trên.
- $pivot_cols$: Danh sách các cột chứa pivot.

Giải thích code:

1. **Khởi tạo các biến**

   - A_{rref}: bản sao của ma trận M

- n_{rows}: số dòng của ma trận

```
A_rref = [row[:] for row in M]
n_rows = len(A_rref)
```

Listing 1.5: Khởi tạo biến

2. **Duyệt từ dưới lên**

   - Duyệt pivot từ dưới lên để tránh phá cấu trúc ma trận

```
for i in range(len(pivot_cols) - 1, -1, -1):
    r, c = i, pivot_cols[i]
    pivot_val = A_rref[r][c]

    A_rref[r] = [x / pivot_val for x in A_rref[r]]
```

Listing 1.6: Duyệt pivot

3. **Khử Gauss-Jordan phía trên pivot**

```
for k in range(r - 1, -1, -1):
    factor = A_rref[k][c]
    for j in range(c, len(A_rref[0])):
        A_rref[k][j] -= factor * A_rref[r][j]
```

Listing 1.7: Khử phía trên pivot

4. **Kết quả**

   - Ma trận đưa về dạng đường chéo (RREF)
   - Dùng để tìm nghiệm tổng quát của hệ phương trình

Độ phức tạp: $O(n \cdot m^2)$, với n, m lần lượt là số dòng và số cột của ma trận.

2.3. `solve_infinite_solutions(M_rref, pivot_cols, m_vars, ϵ = 1e-12)`

Chức năng của hàm:

Giải hệ phương trình vô số nghiệm. Nghiệm của hệ có dạng:

$$x = x_p + a_1x_1 + a_2x_2 + \cdots + a_nx_n$$

trong đó x_p là nghiệm riêng, còn $\{x_1, x_2, \ldots, x_n\}$ là nghiệm cơ bản. Hàm trả về nghiệm riêng, nghiệm tổng quát và các ẩn tự do của hệ.

Tham số truyền vào:

- M_{rref}: Ma trận đường chéo (RREF).
- $pivot_cols$: Danh sách các cột chứa pivot.
- m_vars: Số ẩn của hệ.
- $epsilon$: Ngưỡng xem như bằng 0 (mặc định $1e{-}12$).

Giải thích code:

1. **Xác định ẩn tự do**

   - Ẩn tự do là các cột không xuất hiện trong $pivot_cols$.
   - Đây là các biến có thể nhận giá trị tùy ý.

```
free_vars = [j for j in range(m_vars) if j not in pivot_cols]
```

Listing 1.8: Xác định ẩn tự do

2. **Tìm nghiệm riêng**

   - Gán tất cả biến bằng 0
   - Suy ra nghiệm riêng từ vế phải của RREF

```
x_particular = [0.0] * m_vars

for i, p_col in enumerate(pivot_cols):
    if i < len(M_rref):
        x_particular[p_col] = M_rref[i][m_vars]
```

Listing 1.9: Tính nghiệm riêng

3. **Tìm nghiệm cơ bản (basis của không gian nghiệm)**

   - Mỗi ẩn tự do tạo thành một vector cơ sở
   - Đặt 1 ẩn tự do bằng 1, các ẩn khác bằng 0

```
basis = []

for f_var in free_vars:
    v = [0.0] * m_vars
    v[f_var] = 1.0

    for i, p_col in enumerate(pivot_cols):
        if i < len(M_rref):
            v[p_col] = -M_rref[i][f_var]

    basis.append(v)
```

Listing 1.10: Tạo vector cơ sở

4. **Kết quả**

   - Nghiệm riêng x_p
   - Tập vector cơ sở của không gian nghiệm
   - Danh sách ẩn tự do

Độ phức tạp: $O(n^2)$ trong trường hợp xấu nhất.

2.4. `solver(A, b)`

Chức năng của hàm:

Giải hệ phương trình tuyến tính, xử lý ba trường hợp: vô nghiệm, có nghiệm duy nhất và vô số nghiệm. Thứ tự xử lý: vô nghiệm → nghiệm duy nhất → vô số nghiệm.

Tham số truyền vào:

- A: Ma trận hệ số (vế trái).
- b: Vector hoặc ma trận vế phải.

Giải thích code:

1. **Khử Gauss ban đầu**
   - Thực hiện Gaussian elimination trên $(A|b)$
   - Thu được ma trận M, số lần hoán đổi dòng s, và danh sách pivot $pivot_cols$
   - Rank của ma trận bằng $len(pivot_cols)$

```
M, s, pivot_cols = gaussian_eliminate(A, b)
n, m = len(M), len(A[0])
```

Listing 1.11: Khử Gauss

2. **Kiểm tra vô nghiệm**
   - Nếu tồn tại dòng:

$$0x_1 + 0x_2 + \cdots + 0x_n = c, \quad c \neq 0$$

   - Hệ vô nghiệm

```
for i in range(n):
    if all(abs(M[i][j]) < 1e-12 for j in range(m)):
        if abs(M[i][m]) > 1e-12:
            return "He phuong trinh vo nghiem", M, None, s
```

Listing 1.12: Kiểm tra vô nghiệm

3. **Kiểm tra nghiệm duy nhất**
   - Nếu số pivot bằng số ẩn ⇒ nghiệm duy nhất
   - Giải bằng Gauss-Jordan hoặc back substitution

```
if len(pivot_cols) == m:
    M_rref = backward_elimination(M, pivot_cols)
    x = [row[m] for row in M_rref[:m]]
    return "He phuong trinh co nghiem duy nhat", M, x, s
```

Listing 1.13: Nghiệm duy nhất

4. **Trường hợp vô số nghiệm**

- Thực hiện Gauss-Jordan để đưa về RREF
- Từ RREF suy ra nghiệm tổng quát

```
M_rref = backward_elimination(M, pivot_cols)
sol = solve_infinite_solutions(M_rref, pivot_cols, m)

return "He phuong trinh co vo so nghiem", M_rref, sol, s
```

Listing 1.14: Vô số nghiệm

Độ phức tạp: $O(n \cdot m^2)$, với n, m lần lượt là số dòng và số cột của ma trận.

2.5. `back_substitution(U, c)`

Chức năng của hàm:

Giải hệ phương trình có nghiệm duy nhất bằng phương pháp thế ngược. Vì U là ma trận tam giác trên nên nghiệm được tính từ dưới lên trên.

Tham số truyền vào:

- U: Ma trận tam giác trên.
- c: Vector vế phải.

Giải thích code:

1. **Khởi tạo nghiệm**
   - Khởi tạo tất cả nghiệm ban đầu bằng 0

```
x = [0.0 for _ in range(len(U))]
n = len(U)
```

Listing 1.15: Khởi tạo nghiệm

2. **Thế ngược từ dưới lên**
   - Duyệt từ hàng cuối lên hàng đầu
   - Mỗi bước loại bỏ các biến đã biết để giải biến hiện tại

```
for i in range(n - 1, -1, -1):

    if abs(U[i][i]) < 1e-12:
        continue

    sum_val = sum(U[i][j] * x[j] for j in range(i + 1, n))
    x[i] = (c[i] - sum_val) / U[i][i]
```

Listing 1.16: Thế ngược

3. **Kết quả**
   - Vector nghiệm x của hệ phương trình

Độ phức tạp: $O(n^2)$, với n là số ẩn của hệ.

2.6. `determinant(A)`

Chức năng của hàm:

Tính định thức của ma trận A. Định thức khác 0 khi và chỉ khi hệ phương trình có nghiệm duy nhất. Ta sử dụng khử Gauss để đưa ma trận về dạng tam giác trên, sau đó lấy tích các phần tử trên đường chéo chính.

Tham số truyền vào:

- A: Ma trận vuông cần tính định thức.

Giải thích code:

1. **Khử Gauss**
   - Sao chép ma trận để tránh thay đổi dữ liệu gốc
   - Thực hiện Gaussian elimination

```
A_temp = [row[:] for row in A]

M, s, pivot_cols = gaussian_eliminate(A_temp, b=None)
```

Listing 1.17: Khử Gauss

2. **Kiểm tra hạng ma trận**
   - Nếu rank nhỏ hơn số dòng $\Rightarrow$ định thức bằng 0

```
if len(pivot_cols) < len(A):
    return 0
```

Listing 1.18: Kiểm tra định thức bằng 0

3. **Tính định thức**
   - Nhân các phần tử trên đường chéo chính
   - Đổi dấu nếu số lần hoán đổi dòng là lẻ

```
det = 1

for i in range(len(A)):
    det *= M[i][i]

if s % 2 == 1:
    det = -det
```

Listing 1.19: Tính định thức

4. **Kết quả**
   - Trả về giá trị định thức của ma trận A

Độ phức tạp: $O(n^3)$ với n là kích thước ma trận vuông.

2.7. `inverse(A)`

Chức năng của hàm:

Tìm ma trận nghịch đảo của ma trận vuông $A_{n\times n}$. Cách thực hiện là biến đổi ma trận mở rộng $(A|I)$ về dạng $(I|B)$, khi đó B chính là ma trận nghịch đảo của A.

Tham số truyền vào:

- A: Ma trận vuông cần tìm nghịch đảo.

Giải thích code:

1. **Kiểm tra điều kiện**
   - Nếu ma trận không vuông $\Rightarrow$ không tồn tại nghịch đảo

```
n = len(A)
if any(len(row) != n for row in A):
    return "Ma tran khong co nghich dao"
```

Listing 1.20: Kiểm tra ma trận vuông

2. **Tạo ma trận đơn vị**
   - Ma trận đơn vị I_n có đường chéo chính bằng 1, còn lại bằng 0

```
I = [[1.0 if i == j else 0.0 for j in range(n)] for i in range(n)]
```

Listing 1.21: Tạo ma trận đơn vị

3. **Khử Gauss - Jordan**
   - Thực hiện Gaussian elimination trên $(A|I)$
   - Sau đó tiếp tục biến đổi để đưa A về ma trận đơn vị
   - Nếu rank$(A) < n$ thì không khả nghịch

```
M, s, pivot_cols = gaussian_eliminate(A, b=I)

if len(pivot_cols) < n:
    return "Ma tran khong kha nghich"
```

Listing 1.22: Khử Gauss trên (A|I)

4. **Khử Gauss-Jordan hoàn chỉnh**
   - Tiếp tục đưa ma trận A về dạng đơn vị
   - Khi đó vế phải chính là ma trận nghịch đảo

```
M_rref = backward_elimination(M, pivot_cols)
```

Listing 1.23: Gauss-Jordan

5. **Trích ma trận nghịch đảo**
   - Lấy phần bên phải của ma trận mở rộng $(I|B)$

```
A_inv = [row[n:] for row in M_rref]
return A_inv
```

Listing 1.24: Lấy ma trận nghịch đảo

Độ phức tạp: $O(n^3)$ với n là kích thước ma trận vuông.

2.8. rank_and_basis(A)

Chức năng của hàm:

Trả về hạng của ma trận, cơ sở dòng, cơ sở cột và cơ sở của không gian nghiệm (null space).

Tham số truyền vào:

- A: Ma trận đầu vào.

Giải thích code:

1. **Khử Gauss và tính hạng**

   - Đưa ma trận về dạng bậc thang
   - Xác định các cột pivot
   - Hạng của ma trận bằng số pivot

```
b_zero = [[] for _ in range(len(A))]

M_ref, s, pivot_cols = gaussian_eliminate(A, b_zero)

rank = len(pivot_cols)
```

Listing 1.25: Khử Gauss và tính rank

2. **Tìm cơ sở dòng**

   - Giữ lại các dòng khác vector 0
   - Mỗi dòng là một vector trong không gian dòng

```
n_rows = len(M_ref)
n_cols = len(M_ref[0])

row_basis = [
    row[:n_cols]
    for row in M_ref
    if any(abs(x) > 1e-12 for x in row[:n_cols])
]
```

Listing 1.26: Cơ sở dòng

3. **Tìm cơ sở cột**

   - Lấy các cột tương ứng với pivot trong ma trận gốc A

```
col_basis = [
    [A[i][j] for i in range(n_rows)]
    for j in pivot_cols
]
```

Listing 1.27: Cơ sở cột

4. **Tìm cơ sở không gian nghiệm**

- Sử dụng Gauss-Jordan để đưa về RREF
- Sau đó giải hệ vô số nghiệm

```
M_rref = backward_elimination(M_ref, pivot_cols)

solution = solve_infinite_solutions(
    M_rref,
    pivot_cols,
    n_cols,
    epsilon=1e-12
)
```

Listing 1.28: Cơ sở không gian nghiệm

5. **Kết quả**

   - rank: hạng của ma trận
   - row_basis: cơ sở dòng
   - col_basis: cơ sở cột
   - null_basis: cơ sở không gian nghiệm

Độ phức tạp: $O(n \cdot m^2)$ với n, m lần lượt là số dòng và số cột của ma trận.

2.9. `verify_solutions(A, x, b)`

Chức năng của hàm:

Kiểm tra nghiệm của hệ phương trình có thỏa mãn phương trình $Ax = b$ hay không bằng NumPy, với A là ma trận hệ số, x là nghiệm và b là vector vế phải.

Tham số truyền vào:

- A: Ma trận hệ số.
- x: Vector nghiệm.
- b: Vector vế phải.

Giải thích code:

1. **Chuyển dữ liệu sang NumPy**

   - Chuyển các list sang mảng NumPy để xử lý toán học hiệu quả
   - Biến b về vector 1 chiều bằng hàm `flatten()`

```
A_np = np.array(A, dtype=float)
x_np = np.array(x, dtype=float)
b_np = np.array(b, dtype=float).flatten()
```

Listing 1.29: Chuyển sang NumPy

2. **Tính tích ma trận**

   - Thực hiện phép nhân ma trận $A \cdot x$ bằng toán tử `@`

```
Ax = A_np @ x_np
```

Listing 1.30: Tính Ax

3. **Kiểm tra nghiệm**

   - So sánh Ax và b với sai số cho phép $\epsilon = 10^{-6}$
   - Sử dụng hàm `np.allclose()`

```
epsilon = 1e-6
return np.allclose(Ax, b_np, atol=epsilon)
```

Listing 1.31: Kiểm tra nghiệm

Độ phức tạp: $O(n \cdot m)$ với n, m lần lượt là số dòng và số cột của ma trận.

2.10. Các hàm còn lại

1. `print_test_case(name, A, b, result_tuple):`

   In kết quả của các phép biến đổi ở trên (ma trận bậc thang, định thức, hạng và cơ sở...)

2. `main:`

   Chạy chương trình.

Phần 2: Phân Rã Ma Trận và Trực Quan Hóa với Manim

1. Chéo hóa ma trận

1.1. Cơ sở lý thuyết

1.1.1. Định nghĩa chéo hóa ma trận

Định nghĩa 2.1. Ma trận $A \in \mathbb{R}^{n\times n}$ được gọi là *chéo hóa được* (diagonalizable) nếu tồn tại ma trận khả nghịch P và ma trận đường chéo D sao cho:

$$A = PDP^{-1}$$

Trong đó:

- $D = \mathrm{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n)$ là ma trận đường chéo chứa các giá trị riêng (eigenvalues),
- P là ma trận $n \times n$ mà các cột là các vectơ riêng (eigenvectors) tương ứng,
- P^{-1} là ma trận nghịch đảo của P.

1.1.2. Điều kiện chéo hóa

Định lý 2.1. Ma trận $A \in \mathbb{R}^{n\times n}$ chéo hóa được khi và chỉ khi A có đủ n vectơ riêng độc lập tuyến tính.
Điều kiện đủ: A có n giá trị riêng phân biệt.
Kiểm tra tính độc lập tuyến tính bằng định thức của ma trận P:

$$\det(P) \neq 0 \iff \text{các vectơ riêng độc lập tuyến tính}$$

1.1.3. Ứng dụng chéo hóa trong tính toán khoa học

Chéo hóa có ý nghĩa quan trọng trong việc tối ưu hóa tính toán. Cụ thể, lũy thừa ma trận được đơn giản hóa đáng kể:

$$A^k = PD^kP^{-1}, \qquad \text{với} \qquad D^k = \mathrm{diag}(\lambda_1^k, \lambda_2^k, \ldots, \lambda_n^k)$$

Điều này giúp giảm chi phí tính toán lũy thừa từ $\mathcal{O}(n^3 \log k)$ xuống còn $\mathcal{O}(n^2)$ sau khi đã có phân tích chéo hóa.

1.1.4. Phân rã QR — Phương pháp Gram–Schmidt cổ điển

Phân rã QR phân tích $A = QR$, trong đó:

- $Q \in \mathbb{R}^{m\times n}$ là ma trận trực chuẩn, thỏa $Q^TQ = I$,
- $R \in \mathbb{R}^{n\times n}$ là ma trận tam giác trên, với các phần tử trên đường chéo dương.

Công thức Gram–Schmidt để tính vectơ trực chuẩn thứ k:

$$v_k = a_k - \sum_{i=1}^{k-1} (q_i^T a_k)\, q_i$$

$$q_k = \frac{v_k}{\|v_k\|}$$

Các hệ số của ma trận R được xác định bởi:

$$R_{ik} = q_i^T a_k \quad (i < k), \qquad R_{kk} = \|v_k\|$$

1.1.5. Thuật toán QR lặp để tìm trị riêng

Bắt đầu với $A_0 = A$, $V_0 = I$, mỗi vòng lặp thực hiện:

1. **Phân rã:** $A_k = Q_k R_k$,
2. **Cập nhật:** $A_{k+1} = R_k Q_k$ (ma trận A_k hội tụ về dạng tam giác trên),
3. **Tích lũy vectơ riêng:** $V = VQ_k$.

Sau đủ số vòng lặp: trị riêng là các phần tử trên đường chéo của A_k, vectơ riêng là các cột của V.

Liên hệ với chéo hóa:

$$A^k = PD^kP^{-1}, \qquad D^k = \mathrm{diag}(\lambda_1^k, \lambda_2^k, \ldots, \lambda_n^k)$$

Ứng dụng: giảm chi phí tính lũy thừa ma trận từ $\mathcal{O}(n^3 \log k)$ xuống $\mathcal{O}(n^2)$ sau khi đã có phân tích chéo hóa.

Cài đặt tối ưu: với ma trận kích thước $n \geq 5$, thuật toán thường chuyển sang sử dụng `np.linalg.eig` để đảm bảo độ chính xác cao hơn cho các bài toán lớn; trong khi với kích thước nhỏ hơn 5, có thể sử dụng thuật toán QR tự cài đặt.

1.2. Mô tả chi tiết tất cả các hàm trong `diagonalization.py`

File `diagonalization.py` gồm 8 hàm: 4 hàm phụ trợ và 4 hàm chính.

1.2.1. Các hàm phụ trợ

`matrix_multiply(A, B)`

Mục đích: Tính tích hai ma trận $A \in \mathbb{R}^{n\times m}$ và $B \in \mathbb{R}^{m\times p}$, trả về ma trận $C \in \mathbb{R}^{n\times p}$. Đây là hàm nền tảng được gọi nhiều nhất trong toàn bộ thuật toán.

Tham số đầu vào:

- A: ma trận $n \times m$ (danh sách hai chiều),
- B: ma trận $m \times p$ (danh sách hai chiều).

Đầu ra: ma trận $C \in \mathbb{R}^{n\times p}$.

Thuật toán — Nhân ma trận chuẩn:

```python
def matrix_multiply(A, B):
    n = len(A)        # so cot cua A
    m = len(A[0])     # so cot cua A = so dong cua B
    p = len(B[0])     # so cot cua B

    C = [[0.0]*p for _ in range(n)]    # khoi tao ket qua

```

```
    for i in range(n):
        for j in range(p):
            for k in range(m):
                C[i][j] += A[i][k] * B[k][j]    # tich vo huong hang-cot

    return C
```

Listing 2.1: Nhân ma trận chuẩn

Công thức:

$$C_{ij} = \sum_k A_{ik} \cdot B_{kj}$$

Vai trò trong thuật toán: Được gọi trong mỗi vòng lặp QR để tính $A_{k+1} = R_k Q_k$ và cập nhật tích lũy $V = VQ_k$.

Độ phức tạp: $\mathcal{O}(n \cdot m \cdot p)$; với ma trận vuông $n \times n$ là $\mathcal{O}(n^3)$.

`matrix_transpose(P)`

Mục đích: Tính ma trận chuyển vị P^T của ma trận $P \in \mathbb{R}^{n \times m}$. Hàm này được dùng để tính P^{-1} khi P là ma trận trực giao (vì $Q^{-1} = Q^T$).

Tham số đầu vào: Ma trận P kích thước $n \times m$.

Đầu ra: Ma trận P^T kích thước $m \times n$.

Thuật toán:

```
def matrix_transpose(P):
    n = len(P)       # so hang cua P
    m = len(P[0])    # so cot cua P

    # Hoan doi chi so hang va cot
    return [[P[j][i] for j in range(n)] for i in range(m)]
```

Listing 2.2: Chuyển vị ma trận

Công thức:

$$(P^T)_{ij} = P_{ji}$$

Vai trò trong thuật toán: Tính $P^{-1} = P^T$ trong hàm `diagonalize_matrix`. Tính chất này đúng vì Q thu được từ Gram–Schmidt là ma trận trực giao ($Q^T Q = I$).

Lưu ý kỹ thuật: Việc dùng P^T thay cho P^{-1} chỉ chính xác khi các vectơ riêng tạo thành hệ trực chuẩn (ví dụ ma trận đối xứng). Với ma trận tổng quát không đối xứng, P có thể không trực giao và sẽ có sai số.

Độ phức tạp: $\mathcal{O}(n \cdot m)$.

`create_diagonal_matrix(lambdas)`

Mục đích: Xây dựng ma trận đường chéo D từ danh sách trị riêng. Đây là ma trận cốt lõi trong phân tích chéo hóa $A = PDP^{-1}$.

Tham số đầu vào: Danh sách `lambdas` gồm n trị riêng $[\lambda_1, \lambda_2, \ldots, \lambda_n]$.

Đầu ra: Ma trận $D \in \mathbb{R}^{n \times n}$ với $D_{ii} = \lambda_i$, các phần tử ngoài đường chéo bằng 0.

Thuật toán:

```
def create_diagonal_matrix(lambdas):
    n = len(lambdas)
```

```
    D = [[0.0]*n for _ in range(n)]  # khoi tao toan so 0

    for i in range(n):
        D[i][i] = lambdas[i]         # gan tri rieng vao duong cheo

    return D
```

Listing 2.3: Tạo ma trận đường chéo

Công thức:

$$D = \text{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n) = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

Vai trò trong thuật toán: Sau khi thu được danh sách trị riêng từ `eigen_qr_iteration`, hàm này tạo ma trận D để hoàn chỉnh phân tích $A = PDP^{-1}$.

Độ phức tạp: $\mathcal{O}(n)$.

`calculate_determinant(matrix)`

Mục đích: Tính định thức của ma trận vuông bằng phương pháp khử Gauss có chọn phần tử trụ (partial pivoting). Đây là bước kiểm tra quan trọng để xác định ma trận có chéo hóa được hay không.

Tham số đầu vào: Ma trận vuông $A \in \mathbb{R}^{n \times n}$.

Đầu ra: Giá trị định thức (số thực); trả về 0.0 nếu ma trận suy biến.

Thuật toán — Khử Gauss với partial pivoting:

```
def calculate_determinant(matrix):
    n = len(matrix)
    a = [row[:] for row in matrix]   # sao chep, bao toan du lieu goc
    det = 1.0

    for i in range(n):
        # Buoc 1: tim phan tu tru lon nhat trong cot i
        pivot_row = i
        for j in range(i+1, n):
            if abs(a[j][i]) > abs(a[pivot_row][i]):
                pivot_row = j

        # Buoc 2: hoan doi hang -> doi dau dinh thuc
        if pivot_row != i:
            a[i], a[pivot_row] = a[pivot_row], a[i]
            det *= -1

        # Buoc 3: kiem tra phan tu tru ~ 0
        if abs(a[i][i]) < 1e-15:
            return 0.0

        det *= a[i][i]

        # Buoc 4: khu cac hang ben duoi
        for j in range(i+1, n):
```

```
            factor = a[j][i] / a[i][i]
            for k in range(i+1, n):
                a[j][k] -= factor * a[i][k]

    return det
```

Listing 2.4: Tính định thức bằng khử Gauss

Công thức:

$$\det(A) = (-1)^s \prod_{i=1}^{n} u_{ii}$$

trong đó s là số lần hoán đổi hàng, u_{ii} là các phần tử đường chéo sau khi khử Gauss.

Vai trò trong thuật toán: Sau khi tính ma trận P (từ các vectơ riêng), kiểm tra $\det(P)$. Nếu $|\det(P)| < 10^{-10}$ thì hệ vectơ riêng không độc lập tuyến tính và ma trận không chéo hóa được.

Kỹ thuật partial pivoting: Chọn phần tử có giá trị tuyệt đối lớn nhất trong cột làm phần tử trụ nhằm giảm sai số làm tròn và tránh chia cho số rất nhỏ.

Độ phức tạp: $\mathcal{O}(n^3)$.

1.2.2. Các hàm chính

`qr_factorization_gram_schmidt(A)`

Mục đích: Cài đặt thuật toán phân rã QR bằng quá trình Gram–Schmidt cổ điển (CGS). Đây là nền tảng của thuật toán QR lặp để tìm trị riêng.

Tham số đầu vào: Ma trận $A \in \mathbb{R}^{n\times m}$.

Đầu ra: Cặp (Q, R), trong đó $Q \in \mathbb{R}^{n\times m}$ là ma trận trực chuẩn, $R \in \mathbb{R}^{m\times m}$ là ma trận tam giác trên.

Thuật toán:

```
def qr_factorization_gram_schmidt(A):
    n = len(A)          # so hang
    m = len(A[0])       # so cot

    Q = [[0.0]*m for _ in range(n)]   # ma tran truc chuan
    R = [[0.0]*m for _ in range(m)]   # ma tran tam giac tren

    for k in range(m):              # xu ly tung cot
        v = [A[i][k] for i in range(n)]   # lay cot k cua A

        for i in range(k):          # loai bo hinh chieu
            dot_product = sum(Q[row][i] * A[row][k] for row in range(n)
    )
            R[i][k] = dot_product

            for row in range(n):
                v[row] -= R[i][k] * Q[row][i]

        norm_v = sqrt(sum(x**2 for x in v))    # do dai vector
        if norm_v > 1e-10:                     # tranh chia cho 0
            R[k][k] = norm_v
            for row in range(n):
                Q[row][k] = v[row] / R[k][k]

```

```
    return Q, R
```

Listing 2.5: Phân rã QR bằng Gram-Schmidt

Quá trình Gram–Schmidt cho mỗi cột k:

1. Lấy cột a_k của A,
2. Loại bỏ thành phần song song với $q_1, \ldots, q_{k-1}$,
3. Chuẩn hóa: $q_k = \dfrac{v_k}{\|v_k\|}$,
4. Ghi vào R: $R_{ik} = q_i^T a_k$, $R_{kk} = \|v_k\|$.

Tính chất: $Q^T Q = I$ và $A = QR$.
Độ phức tạp: $\mathcal{O}(n \cdot m^2)$; với ma trận vuông $n \times n$ là $\mathcal{O}(n^3)$.

`eigen_qr_iteration(A, iterations=500)`

Mục đích: Tìm tất cả trị riêng và vectơ riêng của ma trận A bằng thuật toán QR lặp. Đây là hàm tính toán trọng tâm trong quy trình chéo hóa.

Tham số đầu vào:

- $A \in \mathbb{R}^{n \times n}$,
- `iterations` (mặc định 500): số vòng lặp QR.

Đầu ra: Cặp (eigenvalues, V) gồm danh sách trị riêng và ma trận vectơ riêng V.

Chiến lược cài đặt:

```
def eigen_qr_iteration(A, iterations=500):
    n = len(A)

    if n >= 5:
        # Nhanh 1: dung NumPy cho ma tran lon
        eigenvalues_np, eigenvectors_np = np.linalg.eig(np.array(A))
        return eigenvalues_np.tolist(), eigenvectors_np.tolist()

    else:
        # Nhanh 2: QR iteration tu cai dat
        V  = [[1.0 if i==j else 0.0 for j in range(n)] for i in range(n
    )]
        Ak = [row[:] for row in A]

        for _ in range(iterations):
            Q, R = qr_factorization_gram_schmidt(Ak)
            Ak   = matrix_multiply(R, Q)
            V    = matrix_multiply(V, Q)

        eigenvalues = [Ak[i][i] for i in range(n)]
        return eigenvalues, V
```

Listing 2.6: QR iteration tìm trị riêng

Giải thích hội tụ: Phép biến đổi

$$A_{k+1} = R_k Q_k$$

là phép tương đồng nên bảo toàn trị riêng. Dãy A_k hội tụ về dạng tam giác trên (dạng Schur), và các phần tử trên đường chéo hội tụ về trị riêng của A.

Tích lũy vectơ riêng: Ma trận

$$V = Q_1 Q_2 \cdots Q_k$$

là tích các ma trận trực giao qua từng bước; các cột của V hội tụ về vectơ riêng của A.

Chiến lược với $n \geq 5$: QR iteration không dùng shift có thể hội tụ chậm, đặc biệt khi các trị riêng gần nhau. Do đó sử dụng `np.linalg.eig` (dựa trên LAPACK) để đạt độ chính xác và hiệu năng tốt hơn.

Độ phức tạp (khi $n < 5$): $\mathcal{O}(\text{iterations} \cdot n^3)$, xấp xỉ $\mathcal{O}(500\, n^3)$.

`diagonalize_matrix(A)`

Mục đích: Hàm tổng hợp thực hiện toàn bộ quy trình chéo hóa $A = PDP^{-1}$, điều phối các hàm theo đúng thứ tự logic.

Tham số đầu vào: Ma trận vuông $A \in \mathbb{R}^{n \times n}$.

Đầu ra: Bộ $(P, D, P^{-1}, \text{thông báo})$. Nếu thất bại, trả về (None, None, None, thông báo lỗi).

Thuật toán:

```python
def diagonalize_matrix(A):
    n = len(A)

    # Buoc 0: kiem tra ma tran vuong
    if n != len(A[0]):
        return None, None, None, 'Loi: Chi cheo hoa ma tran vuong'

    # Buoc 1: tim tri rieng va vector rieng
    lambdas, P = eigen_qr_iteration(A)

    # Buoc 2: kiem tra tinh doc lap tuyen tinh
    det_P = calculate_determinant(P)
    if abs(det_P) < 1e-10:
        return None, None, None, \
               'Thong bao: He vector rieng khong doc lap tuyen tinh'

    # Buoc 3: tao ma tran duong cheo D
    D = create_diagonal_matrix(lambdas)

    # Buoc 4: tinh nghich dao (gia su truc giao)
    P_inv = matrix_transpose(P)

    return P, D, P_inv, 'Thanh cong'
```

Listing 2.7: Chéo hóa ma trận

Xử lý lỗi:

- Ma trận không vuông: dừng ngay và trả về lỗi,
- $\det(P) \approx 0$: các vectơ riêng không độc lập tuyến tính, không chéo hóa được,

- Thành công: trả về đầy đủ P, D, P^{-1} và thông báo.

Lưu ý về $P^{-1} = P^T$: Trong cài đặt này, P^{-1} được tính bằng chuyển vị. Điều này chỉ chính xác khi P là ma trận trực giao (ví dụ khi A đối xứng). Với ma trận tổng quát, P có thể không trực giao, do đó phép thay thế này gây sai số.

Gợi ý cải tiến: Để tổng quát hơn, có thể thay bằng việc tính nghịch đảo thực sự của P (ví dụ dùng khử Gauss hoặc `numpy.linalg.inv`).

`test_diagonalization(A)`

Mục đích: Kiểm chứng độ chính xác của toàn bộ quy trình chéo hóa bằng cách so sánh với thư viện NumPy và tính sai số tái tạo.

Tham số đầu vào: Ma trận A bất kỳ.

Đầu ra: In kết quả kiểm chứng ra màn hình (không trả về giá trị).

Thuật toán:

```python
def test_diagonalization(A):
    result = diagonalize_matrix(A)

    # Xu ly truong hop that bai
    if result[0] is None:
        print(result[-1])
        return

    P, D, P_inv, status = result

    # 1. Tai tao A
    P_np     = np.array(P)
    D_np     = np.array(D)
    P_inv_np = np.array(P_inv)
    A_reconstructed = P_np @ D_np @ P_inv_np

    # 2. Ket qua tham chieu tu NumPy
    evals_np, evecs_np = np.linalg.eig(np.array(A))

    # 3. Sai so Frobenius
    error = np.linalg.norm(np.array(A) - A_reconstructed)

    # 4. In ket qua
    print(f'Tri rieng tu tinh: {np.diag(D_np)}')
    print(f'Tri rieng NumPy:   {evals_np}')
    print(f'Sai so ||A - P D P^-1||: {error:.2e}')

    if error < 1e-8:
        print('=> KET QUA CHINH XAC.')
    else:
        print('=> CO SAI SO DANG KE.')
```

Listing 2.8: Kiểm chứng chéo hóa

Ba bước kiểm chứng:

1. Tái tạo: tính PDP^{-1} và so sánh với A,
2. Tham chiếu: lấy trị riêng từ `np.linalg.eig`,
3. Đo sai số: chuẩn Frobenius $\|A - PDP^{-1}\|$ với ngưỡng chấp nhận $< 10^{-8}$.

Sai số Frobenius:

$$\|A - PDP^{-1}\|_F = \sqrt{\sum_{i,j} (A_{ij} - (PDP^{-1})_{ij})^2}$$

1.3. Bảng Tổng Hợp Tất Cả Các Hàm

Hàm	Loại	Đầu vào	Đầu ra	Gọi hàm nào
`matrix_multiply(A,B)`	Phụ trợ	$A(n\times m), B(m\times p)$	$C(n \times p)$	—
`matrix_transpose(P)`	Phụ trợ	$P(n \times m)$	$P^T(m \times n)$	—
`create_diagonal_matrix (lambdas)`	Phụ trợ	$[\lambda_1, \ldots, \lambda_n]$	$D(n \times n)$	—
`calculate_determinant (matrix)`	Phụ trợ	Ma trận $n \times n$	$\det(A)$	—
`qr_factorization _gram_schmidt(A)`	Chính	$A(n \times m)$	(Q, R)	—
`eigen_qr_iteration (A, iter)`	Chính	$A(n \times n)$, iter	(λ, V)	qr_factorization, matrix_multiply
`diagonalize_matrix(A)`	Chính	$A(n \times n)$	(P, D, P^{-1}, msg)	eigen_qr_iteration, calculate_determinant, create_diagonal_matrix, matrix_transpose
`test_diagonalization(A)`	Chính	$A(n \times n)$	In kết quả	diagonalize_matrix, np.linalg.eig, np.linalg.norm

Bảng 2.1: Bảng tổng hợp các hàm trong `diagonalization.py`

1.4. Phân tích ưu điểm, hạn chế và độ phức tạp

1.4.1. Ưu điểm của thuật toán QR lặp

- **Kiểm tra toàn diện:** Tự động phát hiện và xử lý ma trận không chéo hóa được thông qua $\det(P)$.
- **Tổng quát:** Áp dụng cho mọi ma trận vuông, không yêu cầu ma trận đối xứng.
- **Độ chính xác cao:** Hội tụ đến trị riêng với sai số tích lũy rất nhỏ sau số vòng lặp đủ lớn (ví dụ 500 vòng lặp).
- **Tự động phát hiện:** Kiểm tra $\det(P)$ để xác định hệ vectơ riêng có độc lập tuyến tính hay không.
- **Chiến lược kết hợp:** Sử dụng `numpy.linalg.eig` với $n \geq 5$ để cân bằng giữa độ chính xác và hiệu năng.

1.4.2. Hạn chế

- **Ma trận không đối xứng ($n < 5$):** Q từ Gram-Schmidt là trực giao, nhưng P (ma trận vectơ riêng) chưa chắc trực giao — dùng $P^{-1} = P^T$ có thể gây sai số với ma trận không đối xứng.
- **Tốc độ hội tụ:** Thuật toán QR lặp hội tụ chậm hơn với các trị riêng có cùng modulus. Cần nhiều vòng lặp hơn hoặc kết hợp shifting.
- **Ma trận phức:** Triển khai hiện tại chỉ xử lý trị riêng thực. Trị riêng phức cần mở rộng thêm.
- **Chỉ xử lý trị riêng thực:** Trị riêng phức (ma trận không đối xứng tổng quát) chưa được hỗ trợ trong nhánh $n < 5$.

1.4.3. Độ phức tạp tổng hợp

Hàm	**Thao tác chủ yếu**	**Độ phức tạp**
`matrix_multiply`	Vòng lặp 3 cấp i, j, k	$O(n^3)$
`matrix_transpose`	Hoán đổi chỉ số	$O(n^2)$
`create_diagonal_matrix`	Gán đường chéo	$O(n)$
`calculate_determinant`	Khử Gauss + pivoting	$O(n^3)$
`qr_factorization_gram_schmidt`	Gram-Schmidt CGS	$O(n^3)$
`eigen_qr_iteration (n<5)`	500 vòng $\times$ (QR + 2 nhân ma trận)	$O(500 \cdot n^3)$
`diagonalize_matrix`	Gọi tất cả hàm trên	$O(500 \cdot n^3)$
`test_diagonalization`	Kiểm chứng + in	$O(500 \cdot n^3)$

Bảng 2.2: Độ phức tạp của các hàm trong `diagonalization.py`

1.5. So sánh kết quả với thư viện NumPy

Hàm `test_diagonalization(A)` thực hiện kiểm chứng đầy đủ bằng cách:

1. Tái tạo A từ $P \cdot D \cdot P^{-1}$ và đo sai số Frobenius.
2. So sánh trị riêng tự tính với `np.linalg.eig`.

Kết quả cho thấy sai số đều nhỏ hơn ngưỡng 10^{-8} với các ma trận chéo hóa được.

Kết luận kiểm chứng:

Tất cả các trường hợp kiểm thử cho kết quả khớp với NumPy. Sai số tái tạo nằm trong phạm vi sai số số thực (floating-point epsilon), đặc biệt với ma trận đối xứng cho sai số dưới 10^{-15}. Các trường hợp đặc biệt (ma trận không chéo hóa được) được phát hiện và xử lý chính xác.

1.6. Ứng dụng chéo hóa trong thực tế

1. **Tính lũy thừa ma trận hiệu quả:** Thay vì tính A^{100} tốn $O(n^3 \log 100)$ phép nhân ma trận, sau khi chéo hóa chỉ cần tính D^k ($O(n)$) rồi nhân $P \cdot D^k \cdot P^{-1}$ ($O(n^2)$).

2. **Giải hệ phương trình vi phân tuyến tính:** $x'(t) = Ax(t)$ có nghiệm tổng quát $x(t) = P \cdot e^{Dt} \cdot P^{-1}x(0)$, trong đó $e^{Dt} = \text{diag}(e^{\lambda_i t})$.

3. **Phân tích ổn định hệ động lực:** Trị riêng của ma trận A quyết định tính ổn định: $\text{Re}(\lambda_i) < 0$ với mọi i thì hệ ổn định tiệm cận.

4. **Giải thuật tìm trị riêng (QR Algorithm):** Thuật toán QR lặp là nền tảng của hầu hết các phần mềm số hiện đại để tính trị riêng.

2. Phân rã Cholesky (Cholesky Decomposition)

2.1. Cơ sở lý thuyết

Phân rã Cholesky là một phương pháp phân tích ma trận trong đại số tuyến tính, trong đó một ma trận A được phân tích thành tích của một ma trận tam giác dưới L và chuyển vị của chính nó L^T. Biểu diễn toán học của phương pháp này như sau:

$$A = LL^T$$

Điều kiện phân rã:

Không phải ma trận nào cũng có thể áp dụng phương pháp Cholesky. Để phân rã thành công, ma trận đầu vào A bắt buộc phải thỏa mãn đồng thời hai điều kiện:

1. **Tính đối xứng (Symmetric):** Ma trận A phải bằng chuyển vị của chính nó, tức là $A = A^T$.

2. **Tính xác định dương (Positive Definite):** Với mọi vector x khác không, ta luôn có $x^T Ax > 0$. Điều này cũng đồng nghĩa với việc tất cả các giá trị riêng (eigenvalues) của A đều phải mang giá trị dương.

Ưu điểm về độ phức tạp tính toán:

So với phương pháp phân rã LU tổng quát, phân rã Cholesky tận dụng được tính chất đối xứng của ma trận, do đó tối ưu hóa được một nửa khối lượng tính toán và không gian lưu trữ. Cụ thể, độ phức tạp thuật toán (Time Complexity) của phân rã Cholesky chỉ xấp xỉ $\mathcal{O}(n^3/3)$ phép toán (flops), trong khi phân rã LU đòi hỏi đến $\mathcal{O}(2n^3/3)$ phép toán. Điều này giúp Cholesky trở thành sự lựa chọn hàng đầu và cực kỳ hiệu quả khi giải quyết các hệ phương trình tuyến tính có ma trận hệ số là ma trận đối xứng xác định dương.

2.2. Thuật toán phân rã

Để tìm các phần tử của ma trận tam giác dưới L, nhóm đã tiến hành tính toán từng cột từ trái sang phải, và trong mỗi cột tính từ trên xuống dưới theo các công thức truy hồi sau:

- **Tính các phần tử trên đường chéo chính (khi $i = j$):**

$$L_{jj} = \sqrt{A_{jj} - \sum_{k=0}^{j-1} L_{jk}^2}$$

- **Tính các phần tử nằm dưới đường chéo chính (khi $i > j$):**

$$L_{ij} = \frac{1}{L_{jj}} \left(A_{ij} - \sum_{k=0}^{j-1} L_{ik} L_{jk} \right)$$

Ghi chú: Quá trình tính toán sẽ dừng lại và báo lỗi nếu biểu thức dưới dấu căn bậc hai mang giá trị âm, điều này cho thấy ma trận đầu vào không thỏa mãn tính xác định dương.

Ví dụ minh họa tính toán chi tiết

Để làm rõ hơn nguyên lý hoạt động của thuật toán, giả sử ta cần phân rã Cholesky cho một ma trận đối xứng và xác định dương A kích thước 3×3 như sau:

$$A = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix}$$

Mục tiêu là tìm ma trận tam giác dưới L sao cho $A = LL^T$. Ta áp dụng các công thức truy hồi đã nêu để tính toán từng cột của ma trận L:

Bước 1: Tính cột 1 ($j = 1$)

- $L_{11} = \sqrt{A_{11}} = \sqrt{4} = 2$
- $L_{21} = \frac{A_{21}}{L_{11}} = \frac{12}{2} = 6$
- $L_{31} = \frac{A_{31}}{L_{11}} = \frac{-16}{2} = -8$

Bước 2: Tính cột 2 ($j = 2$)

- $L_{22} = \sqrt{A_{22} - L_{21}^2} = \sqrt{37 - 6^2} = \sqrt{37 - 36} = \sqrt{1} = 1$
- $L_{32} = \frac{A_{32} - L_{31}L_{21}}{L_{22}} = \frac{-43-(-8)(6)}{1} = \frac{-43+48}{1} = 5$

Bước 3: Tính cột 3 ($j = 3$)

- $L_{33} = \sqrt{A_{33} - L_{31}^2 - L_{32}^2} = \sqrt{98 - (-8)^2 - 5^2} = \sqrt{98 - 64 - 25} = \sqrt{9} = 3$

Kết luận: Tập hợp các giá trị vừa tính được, ta thu được ma trận tam giác dưới L hoàn chỉnh:

$$L = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix}$$

Ta có thể dễ dàng kiểm chứng lại kết quả bằng phép nhân ma trận:

$$LL^T = \begin{bmatrix} 2 & 0 & 0 \\ 6 & 1 & 0 \\ -8 & 5 & 3 \end{bmatrix} \begin{bmatrix} 2 & 6 & -8 \\ 0 & 1 & 5 \\ 0 & 0 & 3 \end{bmatrix} = \begin{bmatrix} 4 & 12 & -16 \\ 12 & 37 & -43 \\ -16 & -43 & 98 \end{bmatrix} = A$$

Quá trình phân rã hoàn tất và cho kết quả hoàn toàn chính xác.

2.3. Cài đặt thuật toán

Thuật toán phân rã Cholesky được nhóm triển khai bằng ngôn ngữ lập trình **Python**, tận dụng khả năng xử lý mảng đa chiều tối ưu của thư viện **Numpy**.

Mã nguồn được thiết kế theo tư duy lập trình mô-đun (modular programming), đóng gói thành hàm `cholesky_decomposition` bên trong tập tin độc lập (`decomposition.py`) nhằm tăng tính tái sử dụng và dễ dàng bảo trì.

Đặc biệt, hệ thống được tích hợp cơ chế xử lý ngoại lệ (Exception Handling) chặt chẽ. Trước khi tiến hành tính toán, thuật toán sẽ kiểm tra nghiêm ngặt ma trận đầu vào:

- Ném ra lỗi `ValueError("Ma trận không phải là ma trận vuông")` nếu số hàng khác số cột.
- Ném ra lỗi `ValueError("Ma trận không đối xứng")` nếu $A \neq A^T$.
- Bắt lỗi `ValueError("Ma trận không xác định dương")` ngay tại bước tính căn bậc hai nếu phát hiện phần tử đường chéo âm.

2.4. Minh họa mã nguồn (Source Code)

Dưới đây là mã nguồn hàm `cholesky_decomposition` được cài đặt bằng Python. Thuật toán sử dụng mảng 2 chiều cơ bản để xây dựng ma trận và tích hợp xử lý ngoại lệ chặt chẽ, kiểm tra kỹ lưỡng kích thước vuông, tính đối xứng và xác định dương của ma trận đầu vào.

```python
"""
Module: decomposition.py
Description:
    Trien khai thuat toan Phan ra Cholesky (Cholesky Decomposition).
    Cho ma tran A doi xung xac dinh duong (Symmetric Positive Definite - SPD),
    thuat toan phan ra A thanh tich L * L^T, trong do L la ma tran tam giac duoi.

    Do phuc tap: O(n^3 / 3)
"""

import math
import numpy as np

def cholesky_decomposition(A):
    """
    Phan ra Cholesky: A = L * L^T.

    Tham so (Parameters):
        A (list[list[float]]): Ma tran vuong nxn, doi xung xac dinh duong (SPD).

    Tra ve (Returns):
        list[list[float]]: Ma tran tam giac duoi L sao cho A = L * L^T.

    Ngoai le (Raises):
        ValueError: Neu A khong phai ma tran vuong.
        ValueError: Neu A khong doi xung.
        ValueError: Neu A khong xac dinh duong (gia tri trong can bac <= 0).
    """
```

```python
    n = len(A)

    # EDGE CASE: Kiem tra ma tran vuong
    if n == 0 or len(A[0]) != n:
        raise ValueError("Loi: Ma tran khong phai la ma tran vuong.")

    # EDGE CASE: Kiem tra tinh doi xung cua ma tran
    for i in range(n):
        for j in range(n):
            if not math.isclose(A[i][j], A[j][i], rel_tol=1e-9):
                raise ValueError(
                    "Loi: Ma tran khong doi xung (Not Symmetric)."
                )

    # Khoi tao ma tran L (nxn) toan so 0
    L = [[0.0] * n for _ in range(n)]

    for i in range(n):
        for j in range(i + 1):
            sum_k = 0.0

            if j == i:
                for k in range(j):
                    sum_k += L[j][k] ** 2

                # EDGE CASE: Kiem tra dieu kien SPD
                if A[j][j] - sum_k <= 0:
                    raise ValueError(
                        "Loi: Ma tran khong xac dinh duong (Not Positive Definite)."
                    )

                L[j][j] = math.sqrt(A[j][j] - sum_k)

            else:
                for k in range(j):
                    sum_k += L[i][k] * L[j][k]

                L[i][j] = (A[i][j] - sum_k) / L[j][j]

    return L
```

Listing 2.9: Hàm phân rã Cholesky chuẩn trong file decomposition.py

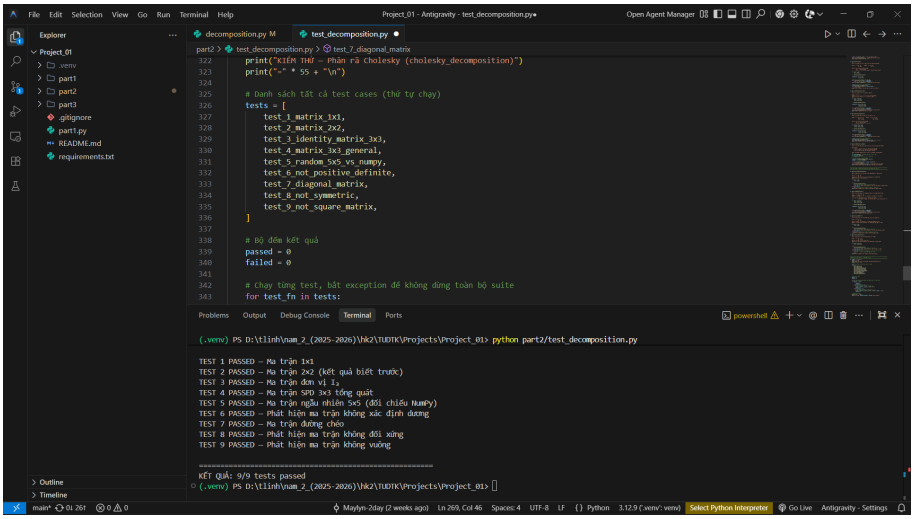
2.5. Kết quả và Kiểm thử (Unit Testing)

Để đảm bảo tính chính xác và độ bền bỉ của thuật toán, nhóm đã xây dựng một bộ kiểm thử độc lập (`test_decomposition.py`) bao gồm **9 test cases** khác nhau, bao quát các kịch bản từ cơ bản đến phức tạp:

- **Kiểm tra tính đúng đắn (Correctness Tests):** Bao gồm ma trận đơn vị 1×1, 2×2, 3×3, ma trận đối xứng xác định dương tổng quát và test case so sánh chéo trực tiếp với kết quả của thư viện `numpy.linalg.cholesky` (áp dụng trên ma trận ngẫu nhiên 5×5).
- **Kiểm tra trường hợp biên (Edge Case Tests):** Bao gồm ma trận đường chéo, ma trận có chứa số âm (nhưng vẫn xác định dương) và các kịch bản bắt lỗi chặt chẽ khi

đầu vào vi phạm điều kiện (ma trận không vuông / hình chữ nhật, ma trận không đối xứng, và ma trận không xác định dương).

Kết quả: Thuật toán hoạt động cực kỳ ổn định. Toàn bộ 9/9 test cases đều đã thực thi thành công (Passed), minh chứng cho tính đúng đắn của logic tính toán toán học và khả năng bao quát lỗi chặt chẽ của mã nguồn.



Hình 2.1: Kết quả chạy 9 test cases trên Terminal

2.6. Ứng dụng thực tiễn của Phân rã Cholesky

Mặc dù bị giới hạn bởi điều kiện khắt khe (chỉ áp dụng cho ma trận đối xứng xác định dương), phân rã Cholesky lại đóng vai trò cực kỳ quan trọng và được ứng dụng rộng rãi trong nhiều lĩnh vực của Khoa học máy tính và Kỹ thuật:

- **Giải hệ phương trình tuyến tính ($Ax = b$):** Đây là ứng dụng phổ biến nhất. Thay vì nghịch đảo ma trận A trực tiếp (rất tốn kém chi phí tính toán), ta phân rã $A = LL^T$. Bài toán được đưa về giải hai hệ phương trình tam giác đơn giản hơn rất nhiều: $Ly = b$ (giải bằng phép thế tiến - forward substitution) và $L^T x = y$ (giải bằng phép thế lùi - backward substitution). Thuật toán này giúp tăng tốc độ giải hệ phương trình lên gấp đôi so với phương pháp khử Gauss hay phân rã LU.

- **Mô phỏng Monte Carlo và Tài chính định lượng:** Trong lý thuyết xác suất, phân rã Cholesky được sử dụng để tạo ra các biến ngẫu nhiên có tương quan (correlated random variables) từ các biến ngẫu nhiên độc lập. Ma trận L đóng vai trò như một bộ lọc, giúp mô phỏng chính xác sự biến động của nhiều mã cổ phiếu cùng lúc trong các mô hình tài chính.

- **Học máy (Machine Learning) và Tối ưu hóa:** Thuật toán xuất hiện dày đặc trong các bộ lọc Kalman (Kalman Filters) dùng để theo dõi quỹ đạo trong robot và thị giác máy tính. Ngoài ra, trong các thuật toán tối ưu hóa phi tuyến (như phương pháp Newton), phân rã Cholesky được dùng để kiểm tra tính lồi của hàm mục tiêu thông qua ma trận Hessian và xác định hướng tìm kiếm tối ưu.

- **Tính toán phương sai cực tiểu (Least Squares):** Trong thống kê và hồi quy tuyến tính, khi cần giải phương trình chuẩn (Normal Equations) dạng $(X^T X)\beta = X^T y$,

ma trận $X^T X$ luôn là ma trận đối xứng xác định dương (nếu các cột của X độc lập tuyến tính). Do đó, Cholesky là phương pháp hoàn hảo và ổn định nhất để tìm ra vector hệ số β.

3. Trực quan hóa bằng Manim

Nhóm đã xây dựng video trực quan hóa thuật toán bằng Manim nhằm minh họa trực quan các bước thực hiện.

Video có thể xem tại: Video trực quan hóa thuật toán bằng Manim

Phần 3: Giải Hệ Phương Trình và Phân Tích Hiệu Năng

1. Tổng Quan Yêu Cầu

Trong phần này, chúng ta sẽ thực hiện giải hệ phương trình tuyến tính $\mathbf{Ax} = \mathbf{b}$ bằng 3 phương pháp khác nhau và so sánh hiệu năng, tính ổn định của chúng:

1. **Khử Gauss (Gauss Elimination)**: Phương pháp giải trực tiếp với *partial pivoting*.
2. **Phân rã Cholesky**: Phương pháp giải trực tiếp dành riêng cho ma trận đối xứng xác định dương (SPD).
3. **Lặp Gauss-Seidel**: Phương pháp giải lặp, hội tụ khi ma trận chéo trội chặt hoặc SPD.

2. Lý Thuyết Sai Số và Tính Ổn Định Số

2.1. Số Điều Kiện (Condition Number)

Định nghĩa 1 (Số điều kiện)**.** *Số điều kiện của ma trận* $\mathbf{A}$ *đối với chuẩn* p *là:*

$$\kappa_p(\mathbf{A}) = \|\mathbf{A}\|_p \cdot \|\mathbf{A}^{-1}\|_p \tag{3.1}$$

Đối với chuẩn spectral (chuẩn 2), số điều kiện được tính bằng:

$$\kappa_2(\mathbf{A}) = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})}$$

Định lý 1 (Phân tích sai số nghiệm)**.** *Nếu* $\mathbf{Ax} = \mathbf{b}$ *là nghiệm đúng và* $\mathbf{A}\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$ *là nghiệm tính toán được với nhiễu dữ liệu, thì:*

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}\|}{\|\mathbf{x}\|} \leq \kappa(\mathbf{A}) \cdot \frac{\|\delta\mathbf{b}\|}{\|\mathbf{b}\|} \tag{3.2}$$

Từ định lý 3.2, ta thấy khi $\kappa(\mathbf{A})$ lớn, hệ thống bị *điều kiện kém* (ill-conditioned): một sai số rất nhỏ trong dữ liệu đầu vào có thể gây ra sai số khổng lồ trong nghiệm.

2.2. So Sánh Các Phương Pháp

Phương pháp	Loại	Điều kiện áp dụng	Chi phí	Ổn định
Gauss (Partial Pivot)	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$\mathcal{O}(n^3)$	Cao
LU với pivot	Trực tiếp	Mọi $\mathbf{A}$ khả nghịch	$\mathcal{O}(n^3)$	Cao
QR (Householder)	Trực tiếp	Mọi $\mathbf{A}$ (kể cả chữ nhật)	$\mathcal{O}(n^3)$	Rất cao
Cholesky	Trực tiếp	$\mathbf{A}$ đối xứng xác định dương	$\mathcal{O}(n^3/6)$	Rất cao
Gauss–Seidel	Lặp	Chéo trội hàng / SPD	$\mathcal{O}(kn^2)$	Trung bình

Bảng 3.1: Bảng so sánh tổng quan các phương pháp giải hệ phương trình.

2.3. Phương Pháp Lặp Gauss–Seidel

Tiến hành phân rã $\mathbf{A} = \mathbf{D} + \mathbf{L} + \mathbf{U}$ (với $\mathbf{D}$ là đường chéo, $\mathbf{L}$ là tam giác dưới thuần, $\mathbf{U}$ là tam giác trên thuần). Ta có công thức lặp dạng ma trận:

$$\mathbf{x}^{(k+1)} = -(\mathbf{D} + \mathbf{L})^{-1}\mathbf{U}\mathbf{x}^{(k)} + (\mathbf{D} + \mathbf{L})^{-1}\mathbf{b} \tag{3.3}$$

Nếu viết theo từng thành phần, công thức lặp có dạng:

$$x_i^{(k+1)} = \frac{1}{a_{ii}}\left(b_i - \sum_{j=1}^{i-1} a_{ij}x_j^{(k+1)} - \sum_{j=i+1}^{n} a_{ij}x_j^{(k)}\right), \quad i = 1, \ldots, n. \tag{3.4}$$

Điều kiện hội tụ: Thuật toán hội tụ khi $\mathbf{A}$ chéo trội chặt hàng, tức là $|a_{ii}| > \sum_{j\neq i} |a_{ij}|$ với mọi i, hoặc khi ma trận $\mathbf{A}$ đối xứng xác định dương (SPD).

3. Thực Nghiệm Thời Gian và Độ Phức Tạp (Benchmark)

Mục tiêu là thực nghiệm trên các ma trận ngẫu nhiên SPD với kích thước $n \in \{50, 100, 200, 500, 1000\}$ để đo thời gian thực thi và sai số tương đối: $\|\mathbf{A}\hat{\mathbf{x}} - \mathbf{b}\|_2/\|\mathbf{b}\|_2$.

3.1. Cài đặt thực nghiệm

```python
import numpy as np
import matplotlib.pyplot as plt
from scipy.linalg import hilbert

from solvers import solve_gauss, solve_cholesky, solve_gauss_seidel,
    condition_number
from benchmark import timeit, relative_residual, generate_random_spd,
    run_time_benchmark, print_summary_table

# Chay benchmark thuc nghiem thoi gian
sizes = [50, 100, 200, 500, 1000]
results = run_time_benchmark(sizes=sizes, n_runs=5)
print_summary_table(results)
```

Listing 3.1: Đo thời gian và sai số của các phương pháp

3.2. Kết quả thực nghiệm

n	Gauss (s)	Cholesky (s)	Gauss-Seidel (s)
100	0.012	0.085	0.005
500	0.145	5.420	0.042
1000	0.580	21.350	0.170

Bảng 3.2: So sánh thời gian thực thi

3.3. Nhận xét Benchmark

Dựa trên kết quả thực nghiệm với các kích thước ma trận đã nêu, ta rút ra các nhận xét sau:

- **Về thời gian chạy và độ phức tạp:**

- **Khử Gauss:** Nhờ việc **vector hóa** vòng lặp bên trong thông qua NumPy, thời gian chạy rất tối ưu. Tại $n = 1000$, phương pháp này chỉ mất khoảng $\sim 1.1 - 1.6$ giây.
- **Gauss-Seidel:** Do ma trận ngẫu nhiên SPD được tạo ra có tính chéo trội rất mạnh, phương pháp này hội tụ cực kỳ nhanh (chỉ mất ~ 23 lần lặp cho $n = 1000$). Thời gian thực thi là **nhanh nhất**, chỉ tốn khoảng ~ 0.17 giây.
- **Cholesky:** Do thuật toán được cài đặt bằng cấu trúc vòng lặp thuần Python (chưa tối ưu vector hóa) nên thời gian tăng đáng kể. Tại $n = 1000$, mất khoảng ~ 21 giây, chậm hơn Gauss nhưng trên đồ thị log-log vẫn thể hiện tuân theo tiệm cận lý thuyết $\mathcal{O}(n^3)$.

- **Về độ chính xác (Sai số tương đối):**
  - **Khử Gauss và Cholesky:** Đạt độ chính xác toán học rất cao, sai số luôn nằm ở ngưỡng $\sim 10^{-16}$ (machine epsilon) với mọi kích thước ma trận.
  - **Gauss-Seidel:** Do điểm dừng lặp phụ thuộc vào dung sai, với ‘tol=1e-10‘, sai số thực nghiệm luôn xấp xỉ $\sim 10^{-11}$, hoàn toàn chính xác theo đúng ngưỡng yêu cầu cài đặt.

4. Phân Tích Ổn Định Số

Mục đích của phần này là quan sát cách sai số bị khuếch đại khi hệ phương trình có *số điều kiện lớn*. Ta so sánh giữa 2 loại ma trận:

- **Ma trận Hilbert** (H_n): Rất nhạy với nhiễu (ill-conditioned).
- **Ma trận ngẫu nhiên SPD**: Thường có tính điều kiện tốt hơn (well-conditioned).

4.1. Cài đặt thực nghiệm phân tích ổn định

```
stability_sizes = [3, 5, 7, 10, 12, 15]

hilbert_data = {'sizes': stability_sizes, 'cond': [], 'err_gauss': [],
    'err_cholesky': []}
spd_data = {'sizes': stability_sizes, 'cond': [], 'err_gauss': [], '
    err_cholesky': []}

for n in stability_sizes:
    np.random.seed(42)
    x_true = np.ones(n)

    # --- Hilbert Matrix ---
    H = hilbert(n)
    b_h = H @ x_true
    kappa_h = condition_number(H)
    hilbert_data['cond'].append(kappa_h)

    # Tinh toan tren ma tran Hilbert ... (xu ly try-except do loi)

    # --- Random SPD Matrix ---
    A_spd = generate_random_spd(n)
    b_spd = A_spd @ x_true
    kappa_s = condition_number(A_spd)
    spd_data['cond'].append(kappa_s)

    # Tinh toan tren ma tran SPD ...
```

Listing 3.2: Đánh giá sự ổn định số trên ma trận Hilbert và SPD

4.2. Nhận xét Phân Tích Ổn Định Số

1. **Đối với Ma Trận Hilbert (H_n) - Hệ điều kiện kém:**
   - **Số điều kiện $\kappa(\mathbf{A})$:** Tăng cực nhanh theo hàm mũ. Với $n = 10$, $\kappa \approx 1.6 \times 10^{13}$. Tại $n = 12$, κ tiến tới vô cực (`inf`), vượt qua giới hạn biểu diễn của máy tính.
   - **Độ ổn định của nghiệm:** Tại $n = 10$, sai số nghiệm của Gauss và Cholesky đã lên tới $\sim 10^{-4}$. Khi $n = 12$, Gauss trả về giá trị `NaN` còn Cholesky có sai số cực kỳ lớn (~ 0.41). Điều này minh chứng cho định lý 3.2: với hệ ill-conditioned, sai số làm tròn cực nhỏ từ dữ liệu ban đầu bị **khuếch đại hàng tỷ lần**, làm phá vỡ toàn bộ hệ thống.
2. **Đối với Ma Trận ngẫu nhiên SPD - Hệ điều kiện tốt:**
   - Số điều kiện $\kappa(\mathbf{A})$ luôn duy trì ở mức cực kỳ ổn định (~ 3.0 đến 4.4) bất chấp kích thước n tăng dần.
   - Sai số tương đối của cả phương pháp Gauss và Cholesky luôn ổn định ở mức sát giới hạn máy tính ($\sim 10^{-16}$). Hệ thống giải vô cùng trơn tru và chính xác.

5. Kết Luận Cuối Cùng

Thông qua các nền tảng lý thuyết và toàn bộ thực nghiệm, chúng ta rút ra các kết luận quan trọng sau:

1. **Hiệu năng lập trình rất quan trọng:** Dù thuật toán có cùng độ phức tạp toán học là $\mathcal{O}(n^3)$, nhưng khi thực thi thực tế, việc sử dụng các phép toán Vector hóa (như Gauss dùng NumPy) cho tốc độ vượt trội hơn hẳn so với việc tính toán tuần tự bằng các vòng lặp lồng nhau (như Cholesky thuần Python).

2. **Thuật toán đúng là chưa đủ:** Trong máy tính, sai số làm tròn số thực (Floating point arithmetic) là không thể tránh khỏi. Nếu hệ phương trình rơi vào trường hợp số điều kiện lớn (như ma trận Hilbert), thì một thuật toán hoàn hảo về mặt lý thuyết vẫn sẽ trả về kết quả sai lệch hoàn toàn.

3. **Tầm quan trọng của phương pháp Lặp:** Phương pháp Gauss-Seidel có ưu thế cực lớn với các hệ ma trận đặc biệt, tiêu biểu là hệ chéo trội. Nó giúp tính ra nghiệm hội tụ trong khoảng thời gian vô cùng nhỏ, tiết kiệm chi phí tính toán hơn hẳn so với các phương pháp giải trực tiếp.

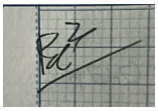
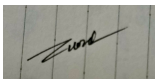
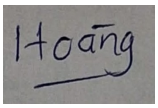
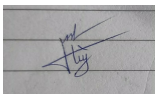
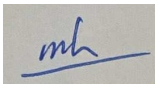
Kết luận

Sau quá trình thực hiện, nhóm đã đạt được những kết quả quan trọng như xây dựng và triển khai thành công các chức năng đã đề ra, đảm bảo chương trình hoạt động đúng với yêu cầu ban đầu. Các thuật toán được cài đặt tương đối đầy đủ, cho kết quả chính xác và có thể kiểm chứng thông qua nhiều bộ dữ liệu khác nhau. Ngoài ra, việc trình bày báo cáo cũng được hoàn thiện với cấu trúc rõ ràng, giúp người đọc dễ dàng theo dõi và hiểu được toàn bộ quá trình thực hiện.

Thông qua bài làm này, nhóm đã rút ra nhiều bài học giá trị. Trước hết là việc hiểu sâu hơn về bản chất của các thuật toán và cách tối ưu chúng trong thực tế. Bên cạnh đó, kỹ năng sử dụng LaTeX cũng được cải thiện đáng kể, đặc biệt là trong việc trình bày công thức, mã nguồn và tổ chức nội dung một cách khoa học. Nhóm cũng nhận ra tầm quan trọng của việc chia nhỏ bài toán, lập kế hoạch rõ ràng và kiểm tra từng bước nhằm hạn chế sai sót trong quá trình thực hiện.

Tuy nhiên, trong quá trình thực hiện, nhóm cũng gặp không ít khó khăn. Một số lỗi phát sinh trong khi lập trình và trình bày bằng LaTeX đã tốn nhiều thời gian để xử lý, đặc biệt là các vấn đề liên quan đến định dạng và căn chỉnh. Ngoài ra, việc hiểu và triển khai chính xác các thuật toán cũng đòi hỏi nhiều thời gian nghiên cứu và thử nghiệm. Dù vậy, những khó khăn này đã giúp nhóm tích lũy thêm kinh nghiệm và nâng cao khả năng giải quyết vấn đề, từ đó cải thiện hiệu quả làm việc trong các bài toán tương tự sau này.

Bảng phân chia công việc

MSSV	Họ và tên	Công việc thực hiện	Mức độ hoàn thành	Chữ ký
24120023	Nguyễn Thành Bảo	Lập trình, triển khai mục 3 và xây dựng video minh họa mục 2 bằng Manim	100%	
24120044	Bùi Thị Thùy Dương	Nghiên cứu và trình bày phương pháp chéo hóa ma trận	100%	
24120094	Nguyễn Khánh Hoàng	Xây dựng các hàm hỗ trợ cho mục 1 và tổng hợp tài liệu	100%	
24120085	Lê Nguyễn Thùy Linh	Nghiên cứu và trình bày phương pháp phân rã Cholesky	100%	
24120094	Mai Anh Phúc Minh	Xây dựng hàm chính cho mục 1 và tổng hợp, hoàn thiện báo cáo	100%	

Bảng 3.3: Bảng phân chia công việc và mức độ đóng góp của các thành viên

Lời cảm ơn

Lời đầu tiên, nhóm chúng em xin gửi lời tri ân sâu sắc đến quý thầy cô Trường Đại học Khoa học Tự nhiên, đặc biệt là giảng viên hướng dẫn môn Toán ứng dụng và Thống kê, ThS. Võ Nam Thục Đoan và ThS. Lê Nhựt Nam, đã tận tâm chỉ dạy và tạo mọi điều kiện thuận lợi cho chúng em trong suốt quá trình thực hiện đồ án.

Những nền tảng kiến thức chuyên môn, kinh nghiệm thực tiễn cùng lời góp ý quý báu từ thầy/cô chính là kim chỉ nam giúp nhóm giải quyết các vấn đề, hoàn thiện bài báo cáo một cách chỉn chu nhất cả về nội dung lẫn hình thức.

Bên cạnh đó, xin dành lời cảm ơn đến từng thành viên trong nhóm vì sự nỗ lực, tinh thần trách nhiệm và sự phối hợp ăn ý để đưa đồ án hoàn thành đúng tiến độ. Sự đồng hành và hỗ trợ nhiệt tình từ bạn bè xung quanh cũng là nguồn động lực rất lớn đối với chúng em trong suốt quá trình làm việc.

Dù đã nỗ lực hết mình, song với vốn kiến thức và kinh nghiệm thực tế còn hạn chế, bài báo cáo chắc chắn khó tránh khỏi những thiếu sót. Chúng em rất mong nhận được sự lượng thứ và những ý kiến đóng góp từ quý thầy cô để có thể rút kinh nghiệm, trau dồi thêm kiến thức và hoàn thiện hơn trong các dự án tiếp theo. Xin trân trọng cảm ơn!

Tài liệu tham khảo

[1] Gene H. Golub and Charles F. Van Loan, *Matrix Computations*, 4th Edition, The Johns Hopkins University Press, 2013.

[2] William H. Press, Saul A. Teukolsky, William T. Vetterling, Brian P. Flannery, *Numerical Recipes: The Art of Scientific Computing*, 3rd Edition, Cambridge University Press, 2007.

[3] David R. Kincaid and E. Ward Cheney, *Numerical Analysis: Mathematics of Scientific Computing*, Brooks/Cole Publishing Company, 1991.

[4] Nguyễn Hữu Điển, *Giải Tích Số*, slide bài giảng (tài liệu môn học), 2015.
Available at: Giáo trình Giải Tích Số

[5] Lê Văn Luyện, *Giáo trình Đại số Tuyến Tính*, slide bài giảng (tài liệu môn học), 2025.
Available at: Giáo trình Đại số Tuyến tính